Architecture 1001: x86-64 Assembly Writing Assembly: GCC

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons "Share Alike" license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

GCC Inline Assembly 1

- GCC uses GNU Assembler (gas) syntax == AT&T syntax
- asm("instructions separated by \n");

```
asm("nop");
asm("movw $1234, %eax");
```

```
asm("movl -0x4(%rbp), %eax\n"
    "cmp $0xdeadbeef,%eax\n"
    "je myLabel\n"
    "xor %eax, %eax\n"
    "myLabel: movw $0xFFFF, %cx\n"
    "and %ecx, %eax");
```

Further reading on gas syntax http://www.ibiblio.org/gferg/ldp/GCC-Inline-Assembly-HOWTO.html

GCC Inline Assembly 2

- How to use C variables (aka "extended asm"):

```
asm ( assembly template :
      output operands   :                 /* optional */
      input operands    :                 /* optional */
      list of clobbered registers         /* optional */
);
```

```
#include <stdio.h>
void main(){
        int myVar = 0xabcd;
        //value into register from C variable
        asm("movl %0, %%eax" : : "r" (myVar) );
        asm("mov $0x1234, %eax");
        //value into C variable from register
        asm("movl %%eax, %0" : "=r" (myVar) );
        printf("myVar = %x\n", myVar);
}
```

Writing Bytes to Get Asm

- You can specify exactly the bytes you want to use by using the ".byte" keyword, to place bytes to be interpreted as code or data
- Examples:
  - asm(".byte 0x90"); is "nop"
  - asm(".byte 0x55"); is "push %rbp"
  - asm(".byte 0x48 ; .byte 0x89 ; .byte 0xE5"); is "mov %rsp, %rbp"

Lab: RTFM && WTFI

- Read The Fun Manual and Write The Fun Instructions!
- Write *raw bytes* to create the following assembly sequence:

```
movw $0xAABBCCDD, %eax
sahf ; What's that? IDK, RTFM!
jz mylabel
and $0x31337, %eax
mylabel:
ret
```